

데이터 분석



Pandas는 row와 column으로 이루어진 데이터 객체를 생성, 수정하기 위한 패키지

- 데이터 분석에 초점이 맞춰져 빅데이터를 다루는데 편리
 - numpy는 수학적 계산을 위한 배열에 초점



Pandas 라이브러리 import 하기

```
# import 할 때는 주로 `pd`를 사용  
import pandas as pd
```

- Pandas에 존재하는 **다양한 함수들을** 이용할 수 있음
- **데이터 구조와 데이터 조작** 기능 제공



1차원 배열과 같은 자료 구조

1차원 배열의 값 뿐 아니라 각 값에 연결된 인덱스 값 저장

	apples
0	3
1	2
2	0
3	1



Series

```
# Series 정의하기
```

```
obj = pd.Series([4, 7, -5])
```

```
0      4  
1      7  
2     -5  
dtype: int64
```

- 0부터 시작하는 **인덱스와 숫자**가 저장됨



Series

Series 정의하기

```
obj = pd.Series([4, 7, -5 ], index = ['2016-01-01', '2016-01-02', '2016-01-03'])
```

2016-01-01	4
2016-01-02	7
2016-01-03	-5

dtype: int64

- 인덱싱 값을 사용자가 설정할 수 있음
- Index에 인덱싱할 값을 넘겨주면 해당 값으로 인덱싱



Series

```
stock0 = pd.Series([10, 20, 30], index=['naver', 'skt', 'LG'])  
stock1 = pd.Series([30, 20, 10], index=['LG', 'naver', 'skt'])  
merge = stock0 + stock1
```

```
LG      60  
naver   30  
skt     30  
dtype: int64
```

- 인덱싱의 순서가 서로 다른 경우에도, **인덱싱이 같은 값끼리 덧셈**을 수행



여러 개의 Series가 모여서 행과 열을 이룬 데이터

	apples	orange	bananas
0	3	5	1
1	2	1	5
2	0	3	4
3	1	4	3



DataFrame

```
raw_data = {'col0': [1, 2, 3, 4],  
            'col1': [10, 20, 30, 40],  
            'col2': [100, 200, 300, 400]}  
data = DataFrame(raw_data)
```

- DataFrame 객체를 생성하기 위해서는 **딕셔너리**를 사용
- **각 컬럼에 저장된 데이터를 사용하여 DataFrame 객체로 생성**



DataFrame

	col0	col1	col2
0	1	10	100
1	2	20	200
2	3	30	300
3	4	40	400

- `'col0'`, `'col1'`, `'col2'`은 DataFrame의 **각 열을 인덱싱** 하는데 사용
- **행 방향**으로는 Series와 유사하게 **정수값으로 자동으로 인덱싱**



DataFrame

```
daeshin = {'open': [11650, 11100, 11200, 11100, 11000],  
           'high': [12100, 11800, 11200, 11100, 11150],  
           'low': [11600, 11050, 10900, 10950, 10900],  
           'close': [11900, 11600, 11000, 11100, 11050]}  
daeshin_day = DataFrame(daeshin, index = [0,2,4,6,8], columns=['open','high','low','close'])
```

- 'col0', 'col1', 'col2'은 DataFrame의 각 열을 인덱싱 하는데 사용
- 행 방향으로는 Series와 유사하게 정수값으로 자동으로 인덱싱



DataFrame

```
a = [[1, 2], [3, 4]]  
df = DataFrame(a, index = [0, 1], columns = ['a', 'b'])
```

	a	b
0	1	2
1	3	4

- **인덱싱에 사용할 값**을 만든 후, DataFrame 객체 생성 시점에 지정



라벨 기반 인덱스를 참조하는 인덱싱/슬라이싱

- **.loc[idx]**
 - DataFrame df에서 **idx 라벨에 해당하는 값**을 추출
- **.loc[idx0 : idx2]**
 - DataFrame df에서 **idx0부터 idx2 라벨에 해당하는 값**들을 추출
 - 슬라이싱 시, **종료 라벨을 포함함**



인덱싱 & 슬라이싱 - loc()

```
data = {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]}  
df = pd.DataFrame(data, index=['a', 'b', 'c'])  
print(df.loc['a':'b', ['A', 'C'])
```

	A	C
a	1	7
b	2	8

- loc 함수를 이용하여 행 라벨 a부터 b까지, 열 라벨 A와, C까지 출력한 결과



정수 기반 인덱스를 참조하는 인덱싱

- **.iloc[idx]**
 - DataFrame df에서 **idx** 인덱스에 해당하는 값들을 추출
- **.iloc[idx0 : idx2]**
 - DataFrame df에서 **idx0**부터 **idx2** 직전 인덱스에 해당하는 값들을 추출
 - 슬라이싱 시, **종료 인덱스를 포함하지 않음**



인덱싱 & 슬라이싱 - iloc()

```
data = {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]}  
df = pd.DataFrame(data, index=['a', 'b', 'c'])  
print(df.iloc[0:2, [0,2]])
```

	A	C
a	1	7
b	2	8

- iloc 함수를 이용하여 행 인덱스 0부터 2 전까지, 열 인덱스 0와, 2까지 출력한 결과



DataFrame에서 행 또는 열을 삭제하기 위한 함수

- **.drop(labels, axis, inplace)**
 - labels = 삭제할 행 또는 열의 이름
 - axis = 삭제할 대상의 축 {0 : 행, 1 : 열}
 - inplace = 원본 객체 수정 여부(default=False)



데이터 삭제 – drop()

```
data = {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]}  
df = pd.DataFrame(data, index=['a', 'b', 'c'])  
df_dropped = df.drop(labels='a')
```

	A	B	C
b	2	5	8
c	3	6	9

- drop 함수를 이용해 a 행을 삭제한 출력 결과



데이터 삭제 – drop()

```
data = {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]}  
df = pd.DataFrame(data, index=['a', 'b', 'c'])  
df_dropped = df.drop(labels='A', axis=1)
```

	B	C
a	4	7
b	5	8
c	6	9

- drop 함수를 이용해 A 열을 삭제한 출력 결과



데이터프레임 또는 시리즈의 값을 기준으로 정렬하는데 사용

- `.sort_values(by, axis, ascending, inplace)`
 - `by` = 정렬을 수행할 기준이 되는 열 또는 열의 리스트
 - `axis` = 정렬할 축 {0 : 행, 1 : 열}
 - `ascending` = 오름차순 정렬 여부 (default=True)
 - `inplace` = 원본 객체 수정 여부 (default=False)



데이터 정렬 – sort_values()

```
data = {'A': [3, 2, 1], 'B': [4, 5, 6]}  
df = pd.DataFrame(data)  
sorted_df = df.sort_values(by='A')
```

	A	B
2	1	6
1	2	5
0	3	4

- A열 기준으로 정렬한 출력 결과
- index 순서 또한 바뀌어진 것을 확인할 수 있음



데이터프레임 간의 연산

- `.add(other, axis, fill_value)`
- `.sub(other, axis, fill_value)`
- `.mul(other, axis, fill_value)`
- `.div(other, axis, fill_value)`
 - other = 연산할 데이터프레임 또는 시리즈
 - axis = 수행할 축 지정
 - fill_value = 연산할 데이터프레임이나 시리즈 내 **누락값(NaN)**이 있을 때, 해당 값을 대체 할 값



데이터프레임 저장 및 불러오기

- **.to_csv(data)**
 - 데이터프레임을 csv 파일로 저장해주는 함수
- **.to_excel(data)**
 - 데이터프레임을 excel 파일로 저장해주는 함수
- **read_csv(data)**
 - csv 파일을 데이터프레임 형식으로 불러오는 함수
- **read_excel(data)**
 - excel 파일을 데이터프레임 형식으로 불러오는 함수
- **data = 저장하거나 불러올 데이터 파일의 이름**



데이터를 그래프나 차트 등의 **시각적 형태**로 표현하여 **이해하고 분석하기 쉽게** 만드는 과정

- 정보 전달의 효율
- 패턴, 상관관계 인식
- 비교와 대조 용이



다양한 데이터 시각화 라이브러리 중 가장 많이 사용되는 라이브러리의 일부

- 간단한 선 그래프
- 산점도
- 히스토그램
- 그래프의 모든 측면을 세밀하게 제어 가능



Matplotlib 라이브러리 import 하기

```
# import 할 때는 주로 `plt`를 사용  
import matplotlib.pyplot as plt
```

- Matplotlib 내의 함수를 사용하기 위해서는 **Matplotlib.pyplot**을 import
- 다양한 그래프를 그리는 함수부터 그래프 관련 제어 함수 사용 가능



Matplotlib 기본 그래프 – plot()

```
plt.plot(["Seoul", "Paris", "Seattle"], [30, 25, 55])
plt.xlabel('City')
plt.ylabel('Response')
plt.title('Experiment Result')
plt.show()
```

- **plot(x, y, fmt)**

- 입력하는 데이터의 포인트들을 연결하여 **직선 그래프**를 출력하는 함수
- x = x축에 해당하는 데이터 값의 배열 또는 리스트
- y = y축에 해당하는 데이터 값의 배열 또는 리스트
- fmt = 선의 색, 스타일 등을 지정하는 문자열, 생략 가능



Matplotlib 기본 그래프 – plot()

```
plt.plot(["Seoul", "Paris", "Seattle"], [30, 25, 55])  
plt.xlabel('City')  
plt.ylabel('Response')  
plt.title('Experiment Result')  
plt.show()
```

- xlabel(xlabel)

- xlabel = x축에 표시할 라벨의 텍스트

- ylabel(ylabel)

- ylabel = y축에 표시할 라벨의 텍스트



Matplotlib 기본 그래프 – plot()

```
plt.plot(["Seoul", "Paris", "Seattle"], [30, 25, 55])  
plt.xlabel('City')  
plt.ylabel('Response')  
plt.title('Experiment Result')  
plt.show()
```

- **title(title)**

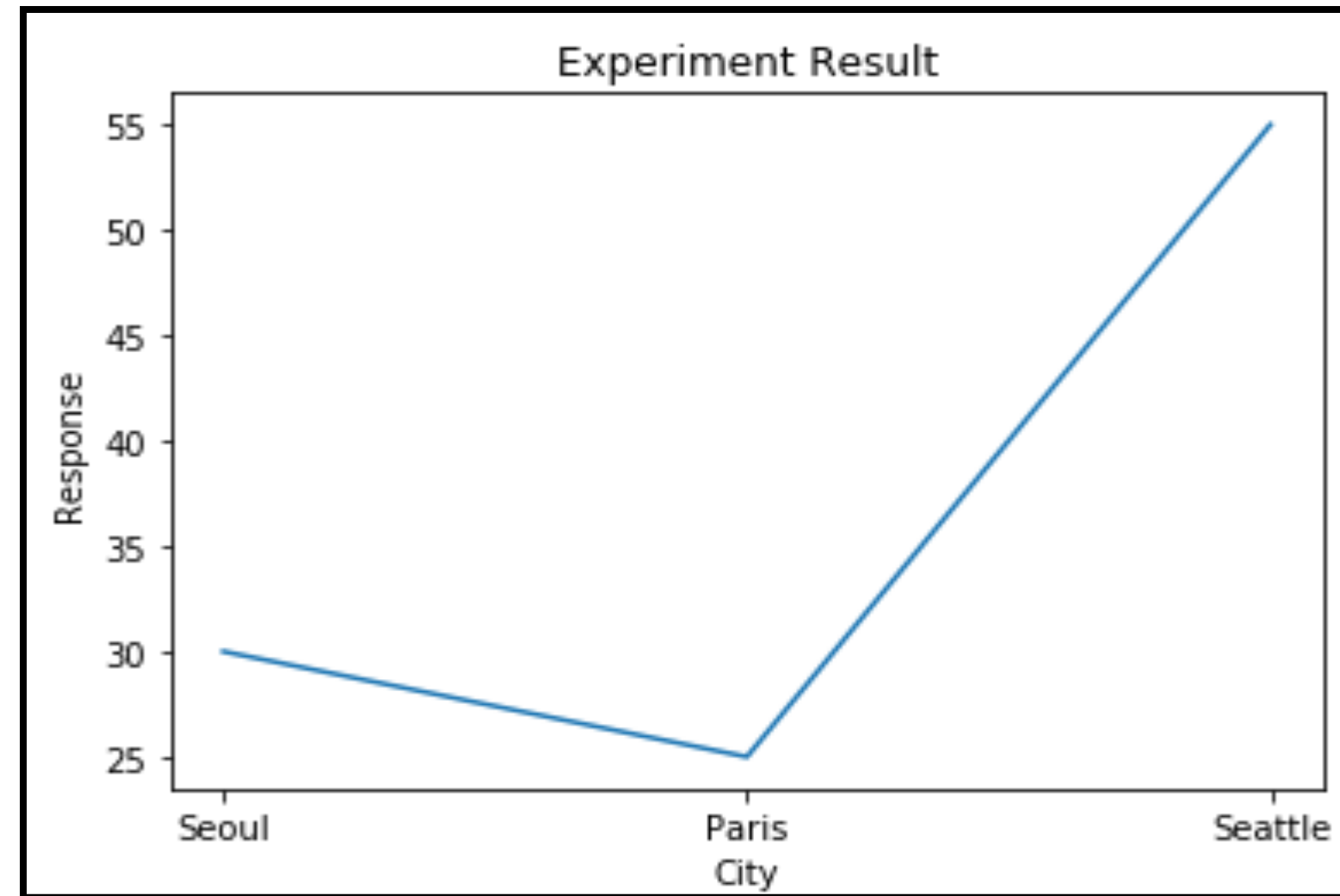
- title = 그래프에 표시할 제목

- **show()**

- 그래프를 화면에 표시하기 위해 반드시 호출되어야 하는 함수
- jupyter notebook과 같은 환경에서는 않아도 출력 가능하나, python idle 환경에선 필수



Matplotlib 기본 그래프 그리기



- 문자열(string)로 입력 받은 경우, 순서에 따라 축에 입력
- (x축 데이터, y축 데이터)인 각 포인트들을 연결한 직선 그래프를 출력

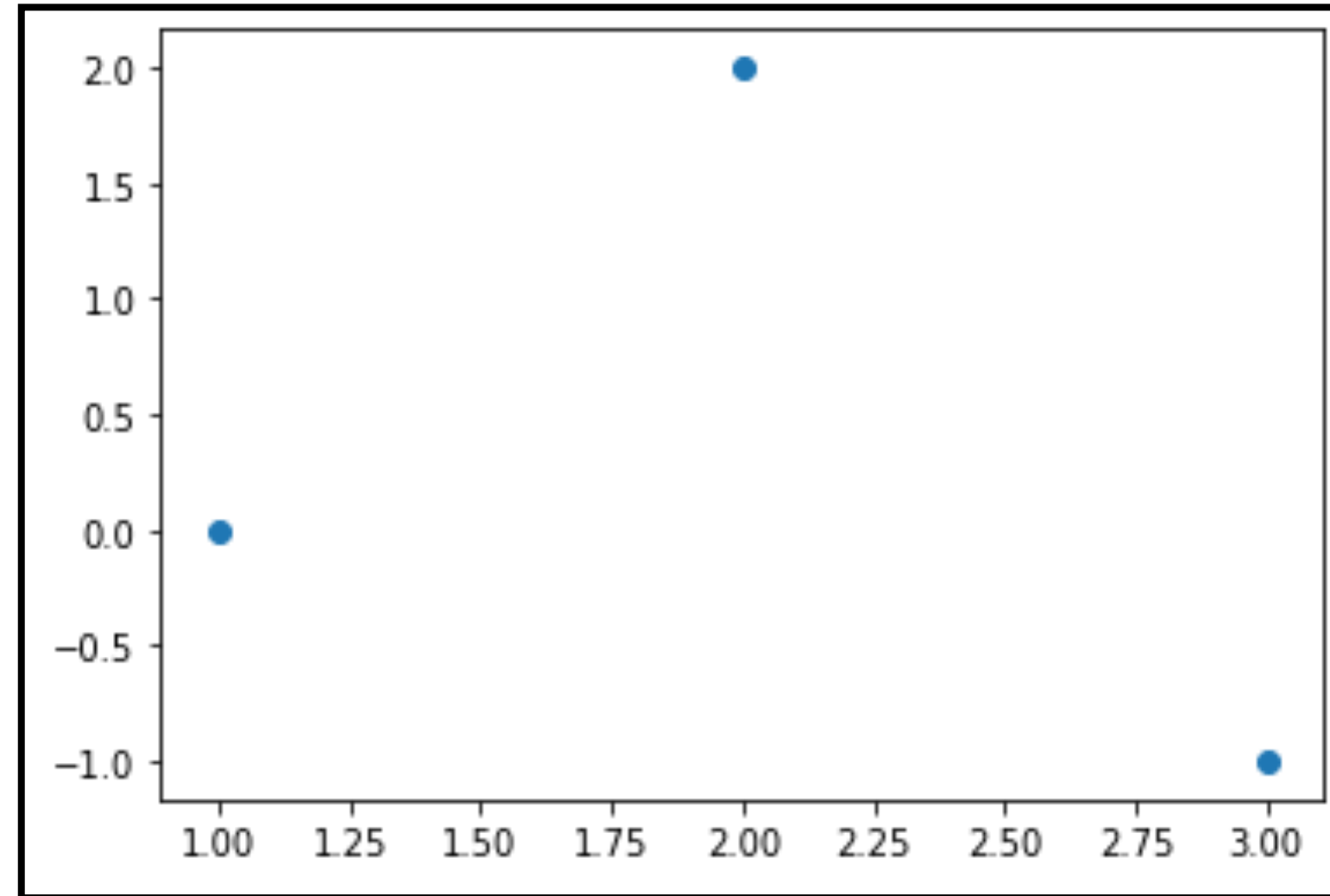
Matplotlib 산점도 그래프 – scatter()

```
plt.scatter([1, 3, 2], [0, -1, 2])  
plt.show()
```

- **scatter(x, y, s, c, marker)**
 - 연결되지 않은 point들을 출력하고 싶을 때 사용
 - x = x축에 해당하는 데이터 값의 배열 또는 리스트
 - y = y축에 해당하는 데이터 값의 배열 또는 리스트
 - s = 점의 크기를 지정하는 스칼라, 배열
 - c = 점의 색상을 지정하는 스칼라, 배열, 또는 컬러맵 (cmap)
 - marker = point의 모양

Matplotlib 산점도 그래프 – scatter()

- cmap
 - 'viridis'
 - 'plasma'
 - 'Blues'
- marker
 - '.' - 작은 점
 - 'o' - 원
 - 'v' - 아래쪽 삼각형
 - 's' - 사각형
 - 'x' - x모양





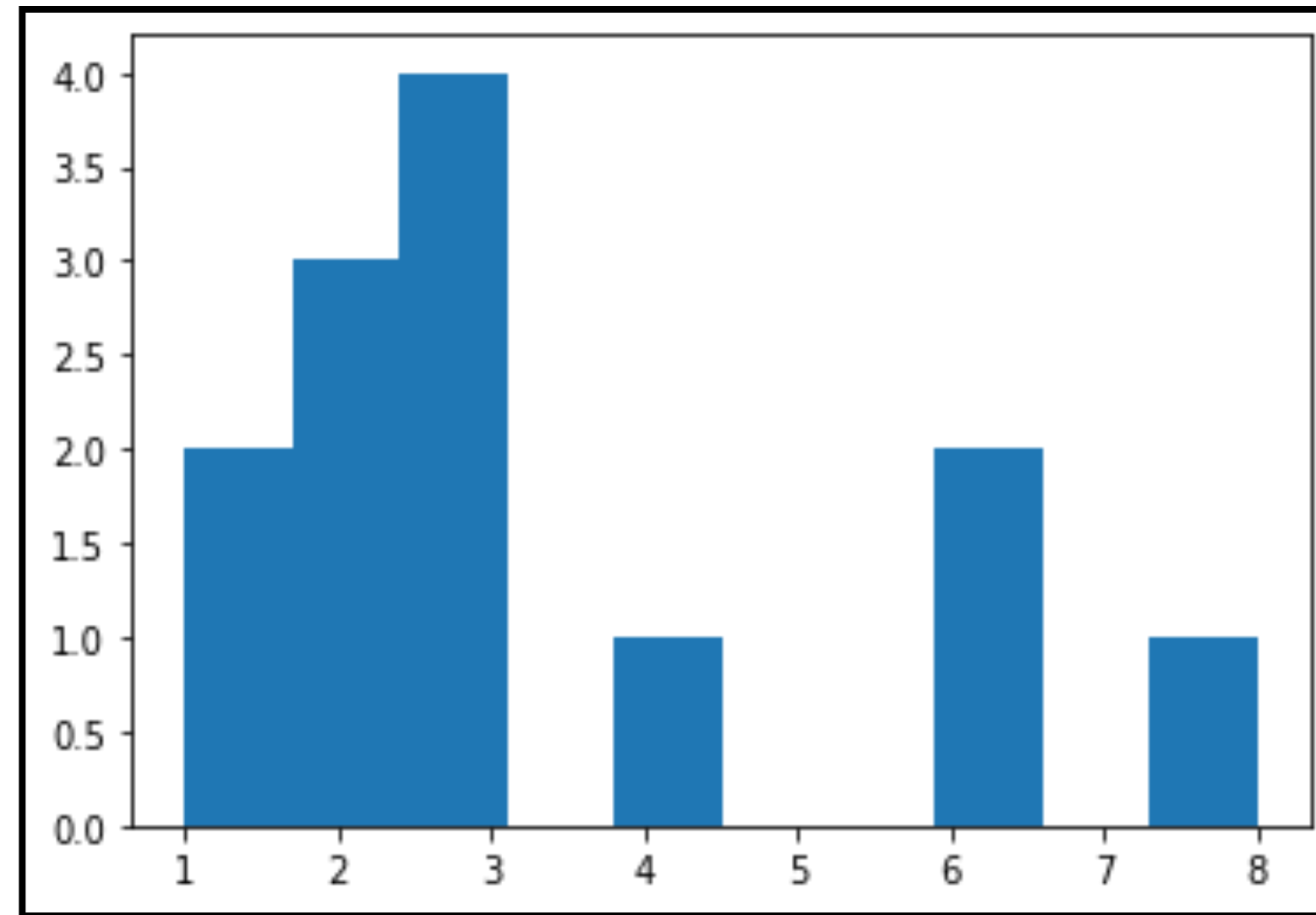
Matplotlib 히스토그램 그래프- hist()

```
plt.hist([1, 3, 2, 2, 3, 3, 3, 1, 4, 8, 2, 6, 6], bins=None)  
plt.show()
```

- hist(x, bins)
 - 히스토그램을 출력하는 함수
 - x = 히스토그램을 그릴 데이터의 배열 또는 리스트
 - bins = 데이터를 나눌 구간의 개수, 구간의 경계값 리스트 (default=10)



Matplotlib 히스토그램 그래프- hist()



- 각 리스트 내 값들의 빈도수를 계산하여 나온 출력 결과
- bins 값을 None으로 지정할 시, 자동으로 적절한 구간 개수를 설정하여 출력



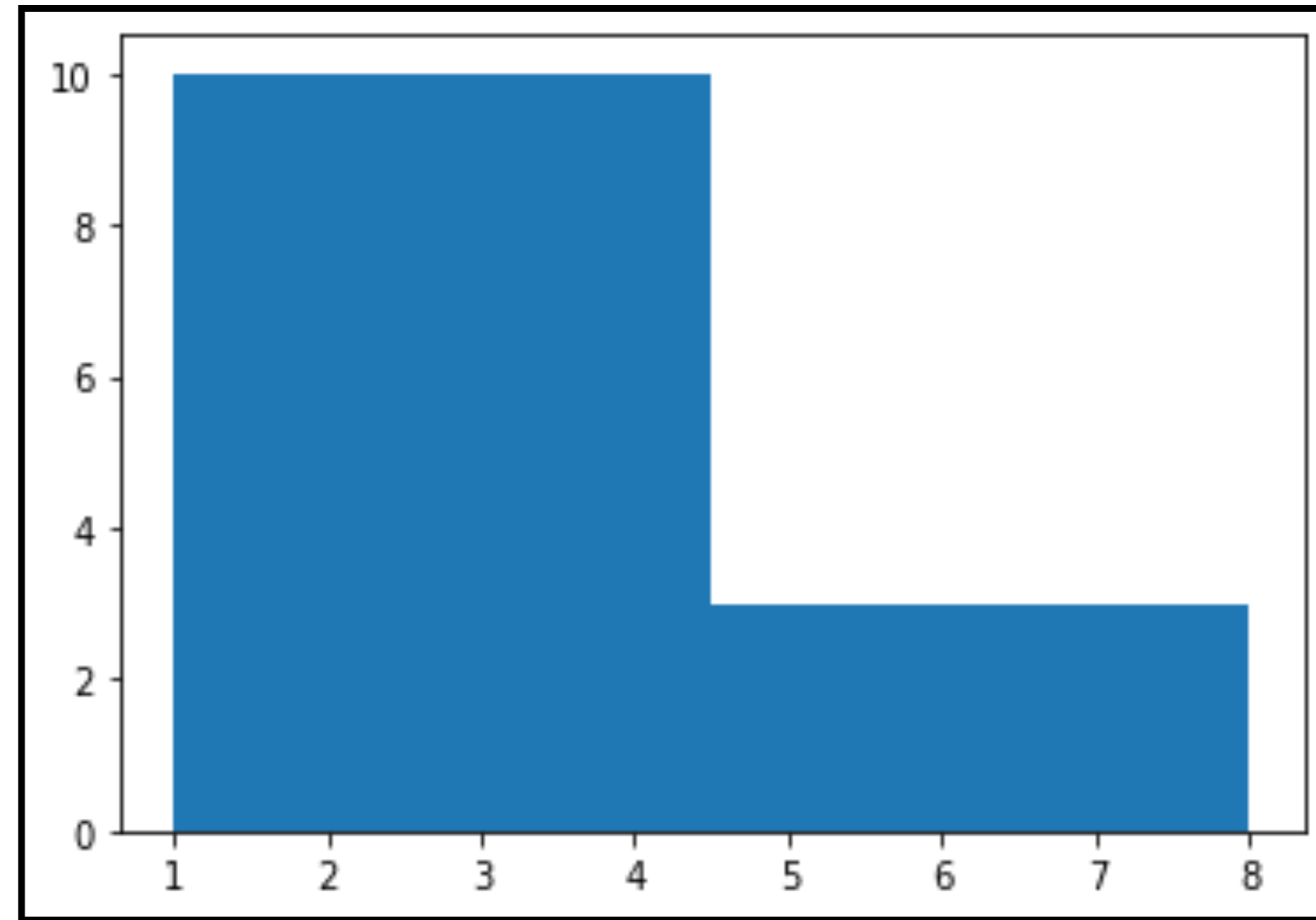
Matplotlib 히스토그램 그래프- hist()

```
plt.hist([1, 3, 2, 2, 3, 3, 3, 1, 4, 8, 2, 6, 6], bins=2)  
plt.show()
```

- bin=2 로 설정 시, 4.5를 기준으로 2개의 range 안에서 개수를 출력
 - 범주를 2개로 나누기 위한 기준 $(1+8)/2=4.5$ 으로 설정



Matplotlib 히스토그램 그래프- hist()



- bins 값을 2로 지정했을 때의 출력 결과
- 4.5를 기준으로 2개의 막대가 생기는 것을 확인할 수 있음



Matplotlib 막대 그래프 – bar()

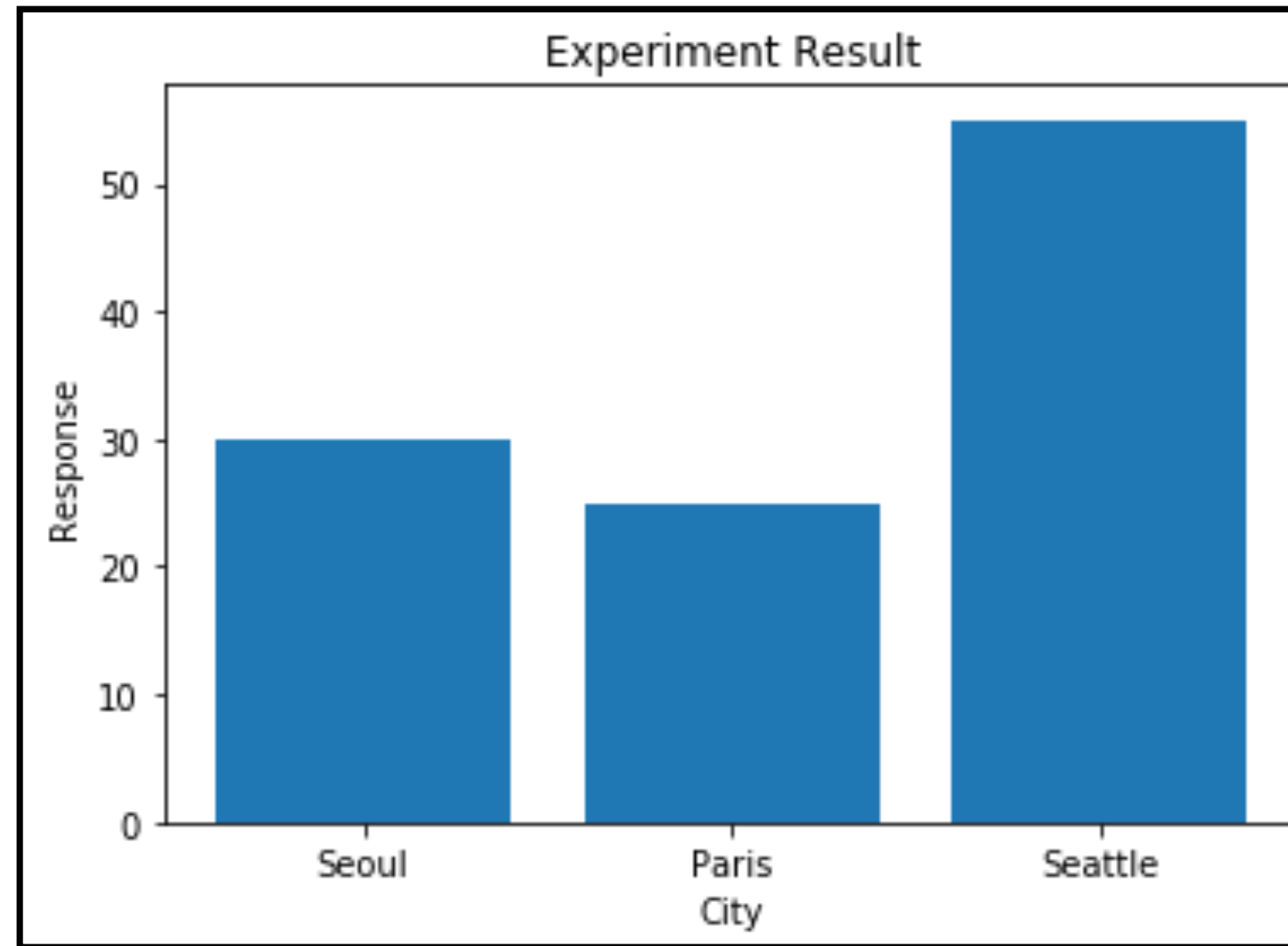
```
plt.bar(["Seoul", "Paris", "Seattle"], [30, 25, 55])  
plt.xlabel('City')  
plt.ylabel('Response')  
plt.title('Experiment Result')  
plt.show()
```

- **bar(x, height, width)**

- 각 x축 포인트에 따른 y축 값을 **막대 그래프**로 그리는 함수
- x = 막대 그래프의 x축에 해당하는 위치를 지정하는 배열 또는 리스트
- height = 막대 그래프의 높이를 지정하는 배열 또는 리스트
- width = 막대의 너비를 지정하는 값 (default=0.8)



Matplotlib 히스토그램 그래프- bar()



- 각 카테고리별 값을 막대 그래프로 시각화한 출력 결과
- plot()과 함수를 제외하고 동일한 코드임을 확인



hist() vs. bar()

hist()

vs.

bar()

데이터 유형	연속형 데이터 시각화	범주형 데이터 시각화
구간	구간 분할 후 빈도수 계산	카테고리별 분할
x축	데이터 값의 범위	범주형 데이터의 카테고리
y축	각 구간 데이터의 빈도 수	카테고리의 값



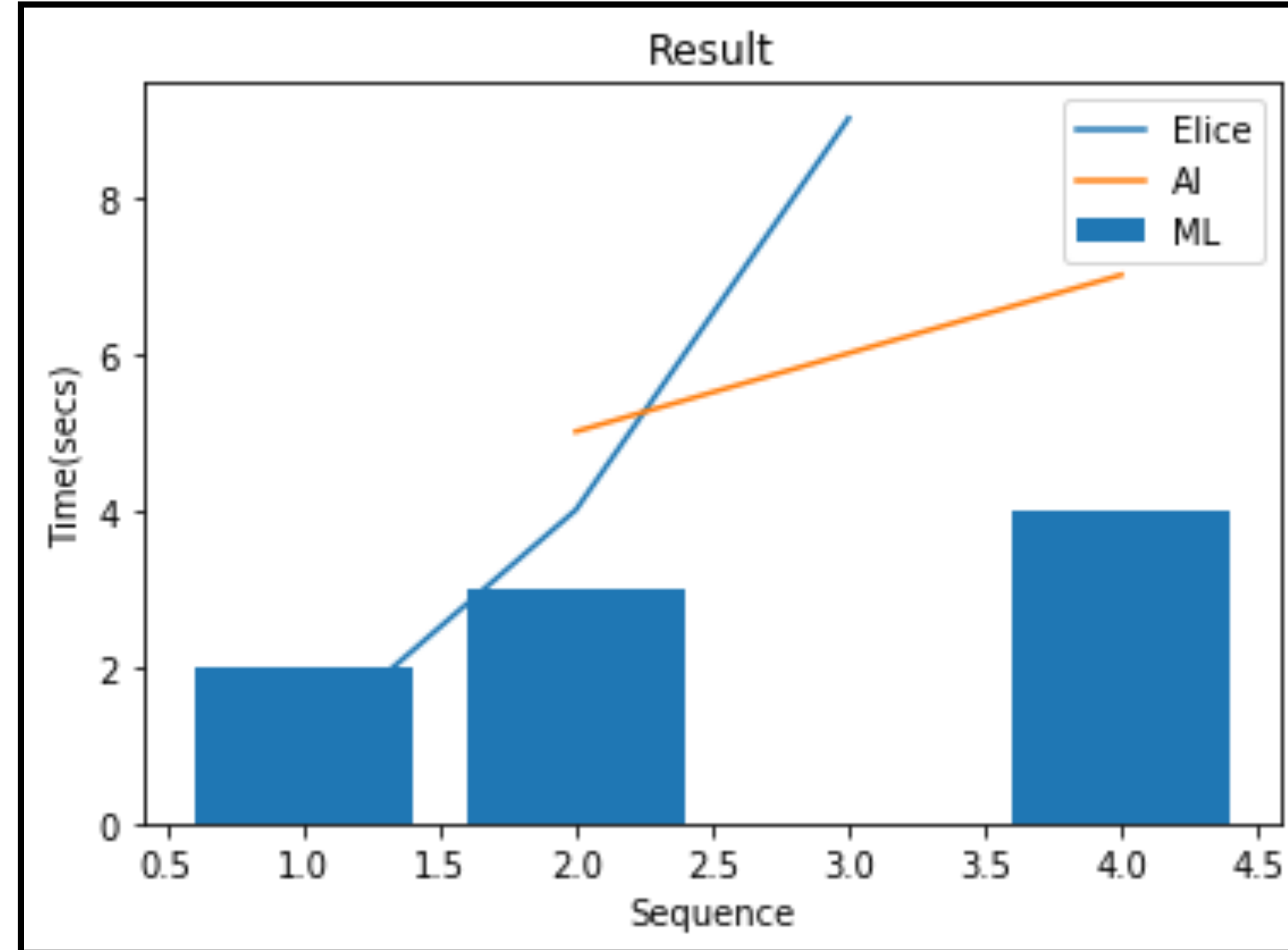
Matplotlib 여러 개의 그래프 출력하기

```
plt.plot([1, 2, 3], [1, 4, 9])
plt.plot([2, 3, 4], [5, 6, 7])
plt.bar([1, 2, 4], [2, 3, 4])
plt.xlabel('Sequence')
plt.ylabel('Time(secs)')
plt.title('Result')
plt.legend(['Elice', 'AI', 'ML'])
plt.show()
```

- 여러 개의 그래프를 동시에 출력하기 위해 여러 개의 그래프 함수를 사용
- legend()
 - 범례를 추가해주는 함수



Matplotlib 여러 개의 그래프 출력하기



- plot 함수 2개, bar 함수 1개를 시각화한 출력 결과

Matplotlib 여러 결과 창 묶어서 출력하기

```
x = np.random.rand(5)
y = np.random.rand(5)

plt.subplot(221)
plt.scatter(x, y, s=80, c='r', marker=">")

plt.subplot(222)
plt.scatter(x, y, s=80, c='b', marker=(5, 0))
...
```

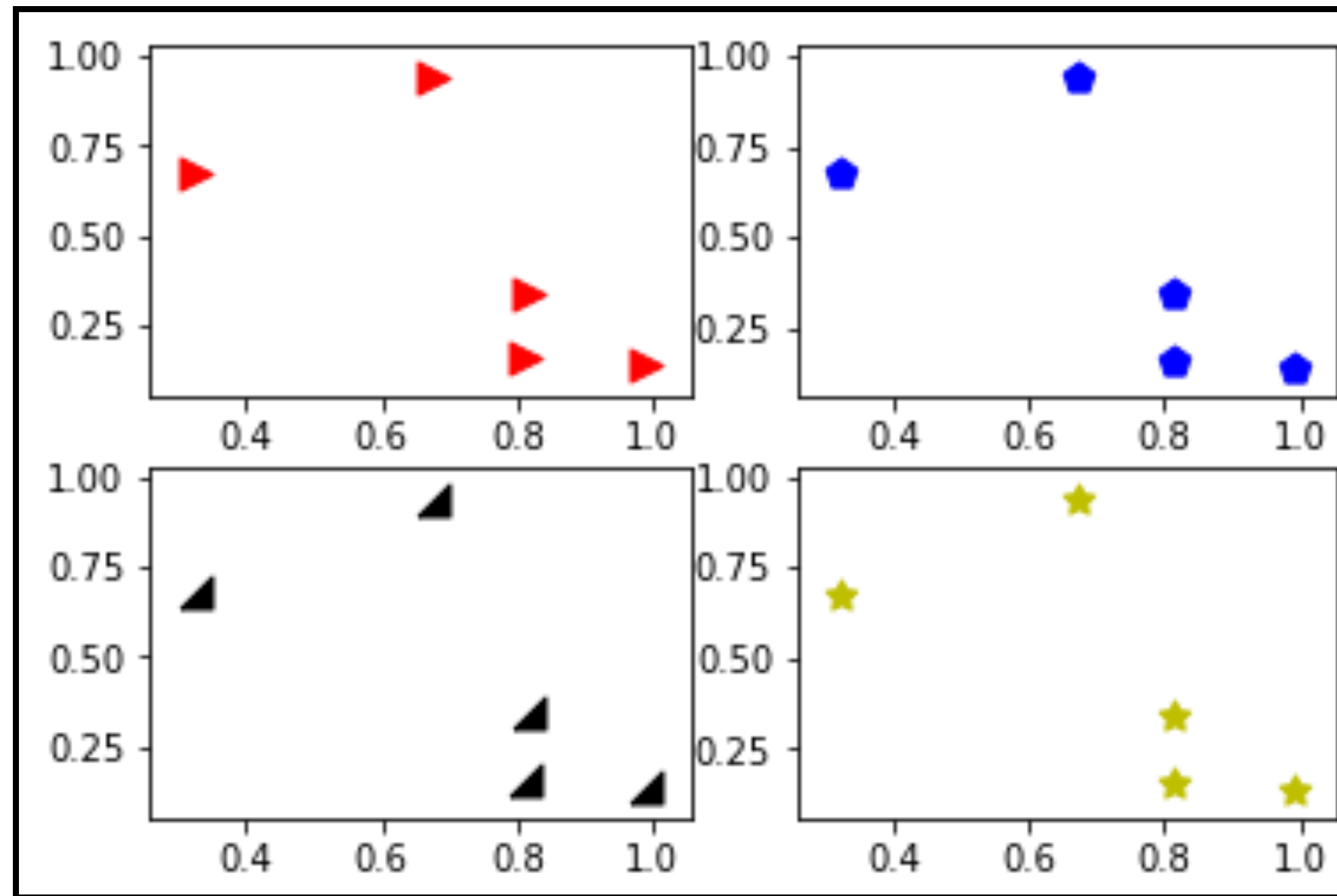
- **subplot(nrows, ncols, index)**
 - 여러 개의 **서브플롯을 생성**하기 위해 사용되는 함수
 - **nrows** = 서브플롯의 행 개수
 - **ncols** = 서브플롯의 열 개수
 - **index** = 서브플롯의 인덱스 지정

Matplotlib 여러 결과 창 묶어서 출력하기

```
...  
verts = np.array([[ -1, -1], [ 1, -1], [ 1, 1], [ -1, 1]])  
plt.subplot(223)  
plt.scatter(x, y, s=80, c='k', marker=verts)  
  
plt.subplot(224)  
plt.scatter(x, y, s=80, c='y', marker=(5, 1))  
  
plt.show()
```

- subplot(221)
 - 2, 2 행렬 출력창에서 1번째 출력창으로 설정
- subplot 코드를 그래프 함수에 앞서 입력해야 해당 그래프의 위치 조정 가능

Matplotlib 여러 결과 창 묶어서 출력하기



- 2, 2 행렬 모양으로 4개의 산점도를 출력한 결과



Matplotlib – fig, axes

```
# a x b 개의 그래프를 그릴 수 있는 figure와 축 설정  
fig, axes = plt.subplots(a, b)
```

- **fig(Figure)**
 - 그림을 나타내는 객체
 - 전체 그림이 존재하는 영역
- **axes**
 - 그림 내의 좌표축을 나타내는 객체
 - 그림 내 여러 개를 가질 수 있음



Matplotlib을 기반으로 다양한 색상 테마, 통계용 차트 기능이 추가된 패키지

- matplotlib 보다 추가된 기능을 사용하여 데이터를 분석하는 경우
- 세련된 테마로 시각화하는 경우



Seaborn 라이브러리 import 하기

```
# import 할 때는 주로 `sns`를 사용  
import seaborn as sns
```

- Seaborn 내 다양한 함수들을 이용하기 위해 seaborn을 import
- 다양한 그래프 그리기 함수 뿐 아니라, 예제 데이터셋을 제공



```
# iris (붓꽃) 데이터 불러오기  
df = sns.load_dataset('iris')
```

- Seaborn의 load_dataset() 함수를 사용하여 데이터를 로드
- 이 외의 데이터를 읽을 시, Pandas 라이브러리의 read_csv(), read_excel과 같은 함수 사용



Seaborn 데이터 불러오기

```
# iris (붓꽃) 데이터 불러오기
```

```
df=sns.load_dataset('iris')
```

```
# 전체 데이터에서 처음 5개의 row 데이터 표시 (내용 확인)
```

```
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa



Seaborn 산점도와 분포 근사선 그래프

```
df = sns.load_dataset('iris')

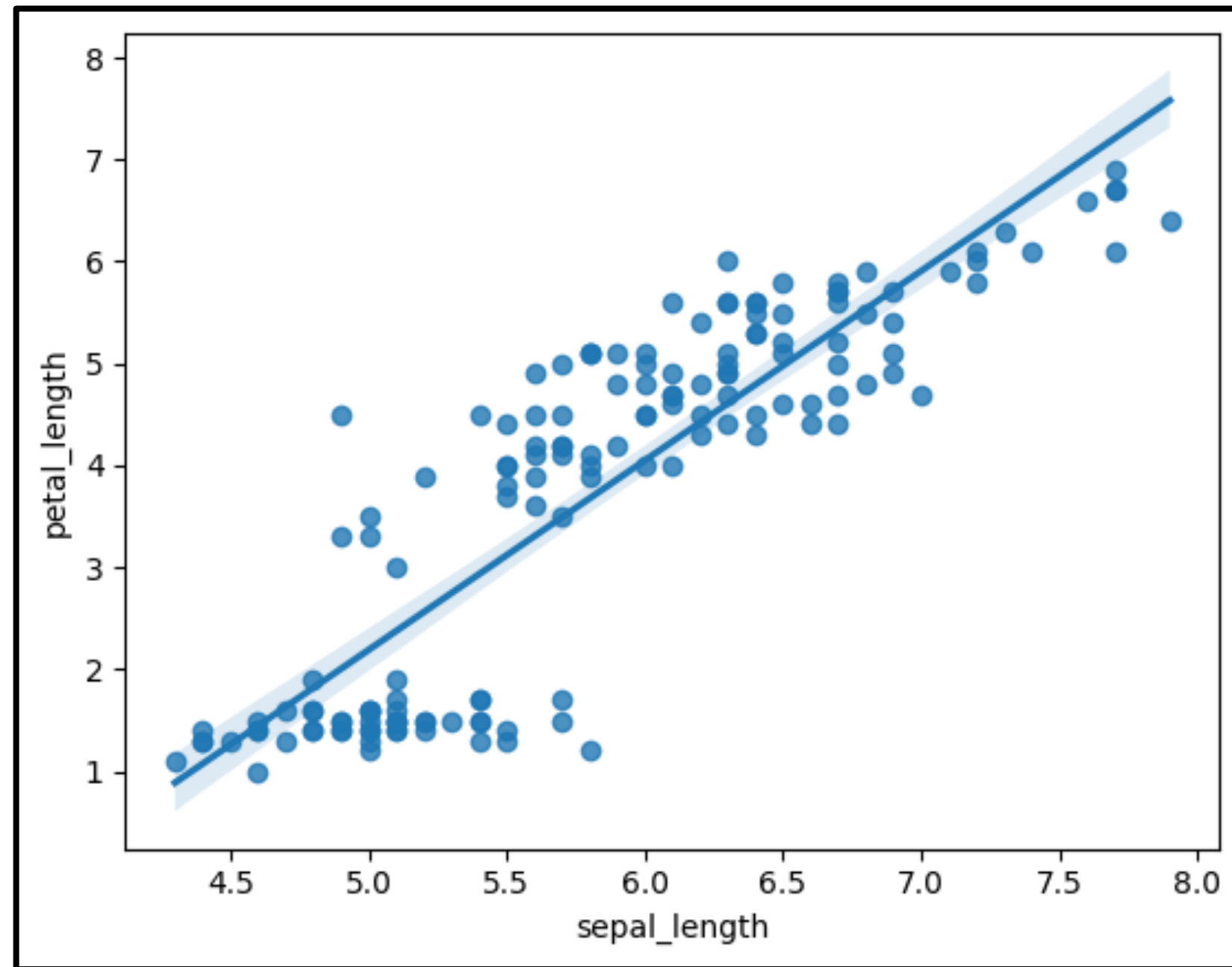
sns.regplot(x="sepal_length", y="petal_length", data = df)
plt.show()
```

- **regplot(x, y, data, color)**

- 데이터의 위치인 **산점도를 출력**할 뿐 아니라, **분포를 근사하는 선도 출력**할 수 있는 함수
- x, y = 그래프에 표시될 데이터의 x축, y축
- data = 사용할 데이터프레임
- color = 그래프 색상 설정



Seaborn 산점도와 분포 근사선 그래프



- 산점도와 분포를 근사하는 선을 함께 출력한 결과



Seaborn 산점도와 분포 근사선 그래프

```
df = sns.load_dataset('iris')

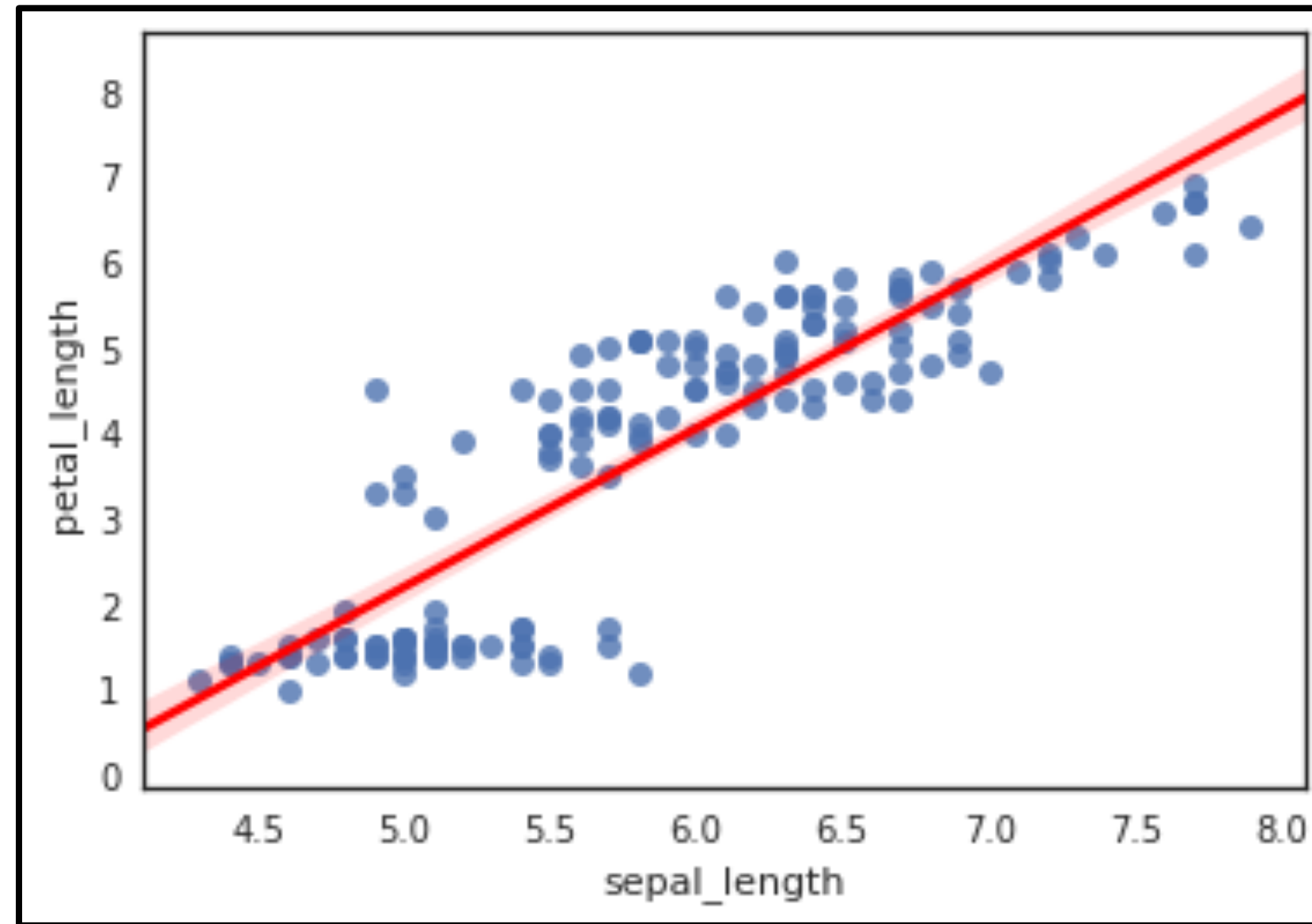
sns.regplot(x="sepal_length", y="petal_length", data = df, line_kws={'color': 'red'})
plt.show()
```

- **line_kws**

- 파라미터 설정을 통한 **근사선** 속성 변경
- color : 근사선의 색상 지정
- linewidth : 근사선의 두께 지정
- linestyle : 근사선의 스타일 지정



Seaborn 산점도와 분포 근사선 그래프



- 근사선의 color 값을 red로 지정한 뒤 출력 결과



Seaborn 산점도와 분포 근사선 그래프

```
df = sns.load_dataset('iris')

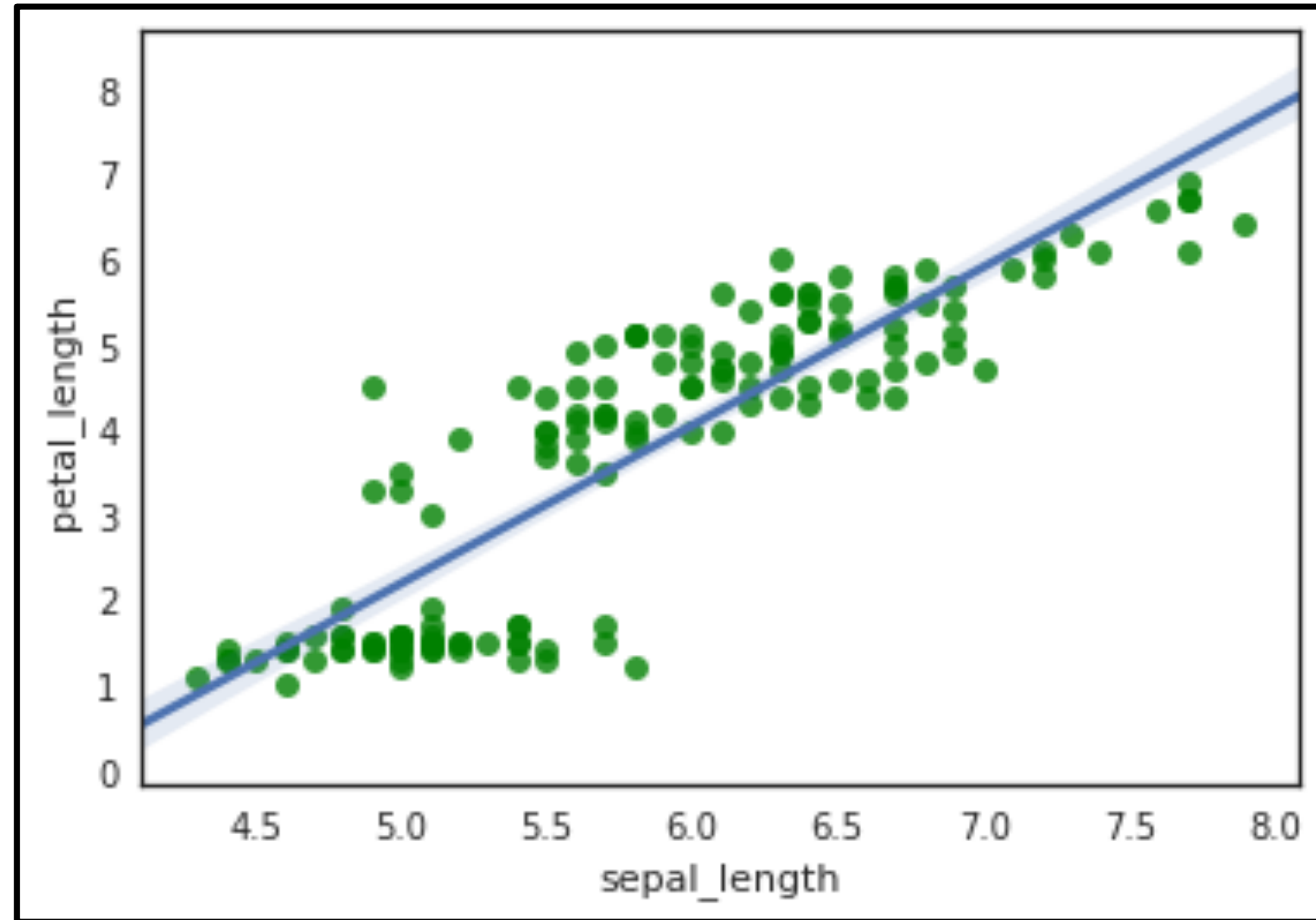
sns.regplot(x="sepal_length", y="petal_length", data = df, scatter_kws={'color': 'green'})
plt.show()
```

- **scatter_kws**

- 파라미터 설정을 통한 산점도 속성 변경
- color : 점의 색상 지정
- s : 점의 크기를 지정
- alpha : 점의 투명도 지정



Seaborn 산점도와 분포 근사선 그래프



- 산점도의 color 값을 green로 지정한 뒤 출력 결과



Seaborn 데이터 불러오기

```
# 타이타닉 데이터 불러오기
```

```
titanic = sns.load_dataset("titanic")
```

```
# 전체 데이터에서 처음 5개의 row 데이터 표시 (내용 확인)
```

```
titanic.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

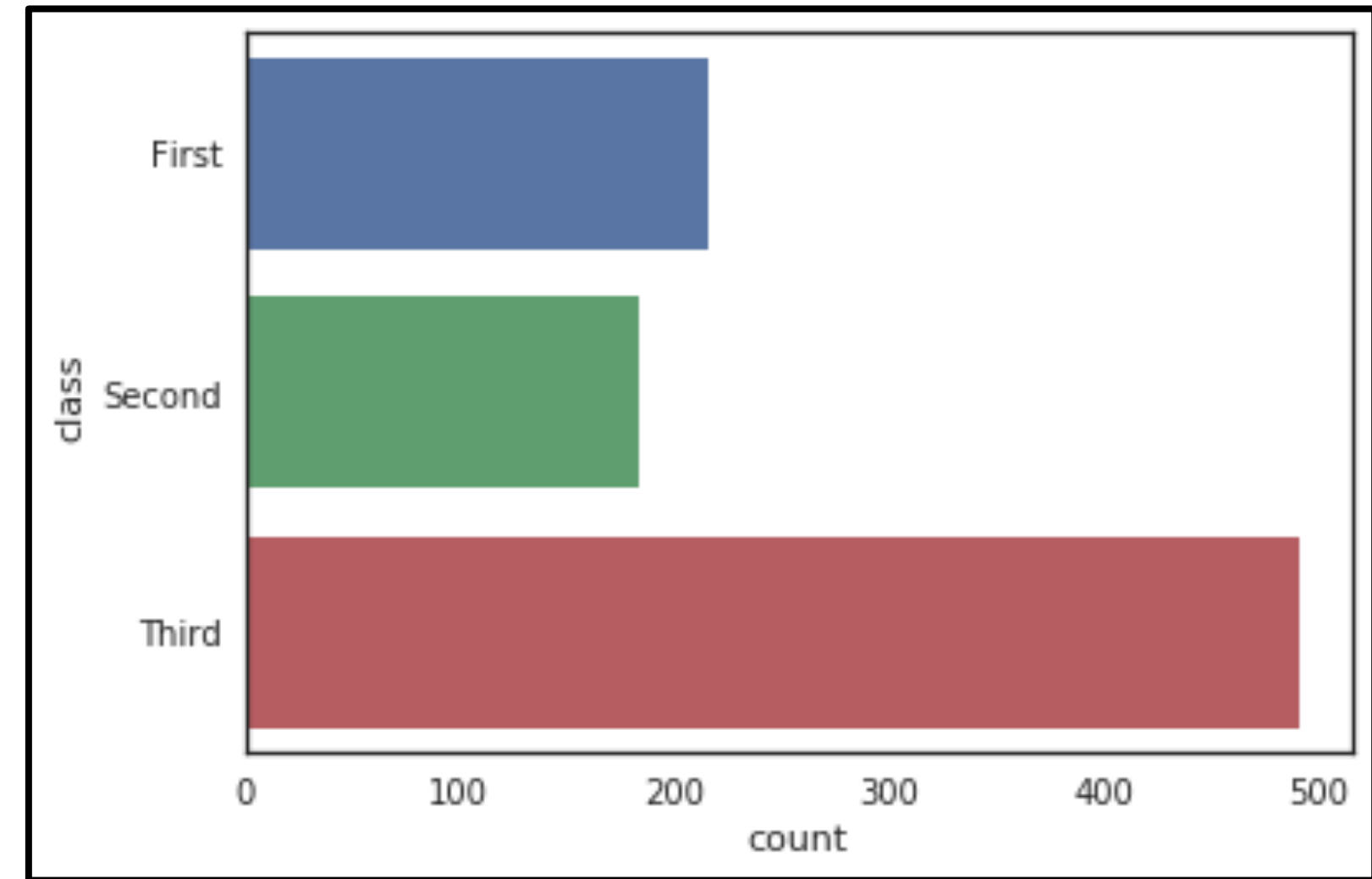
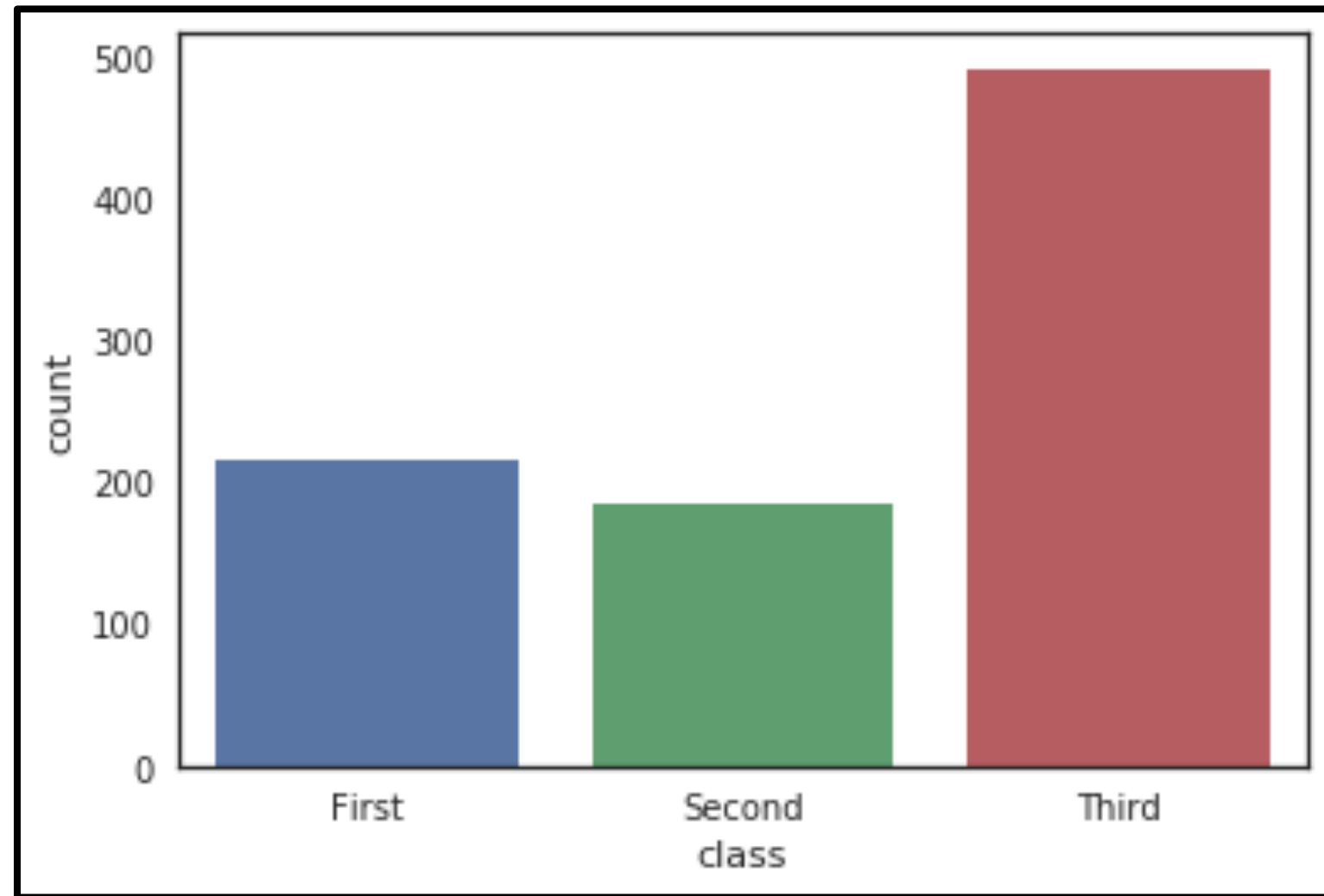


```
sns.countplot(x="class", data=titanic)  
plt.show()
```

- **countplot(x, hue, data)**
 - 범주형 데이터의 빈도를 시각화하는데 사용
 - x = 시각화할 데이터 변수
 - hue : x, y 외 구분할 또 다른 변수 지정
 - data = 사용할 데이터프레임 지정



Seaborn 빈도수 막대 그래프



- class를 기준으로 countplot을 적용한 출력 결과
- x = "class"가 아닌 y = "class"로 지정 시, y축 기준 countplot을 확인할 수 있음



Seaborn 데이터 불러오기

```
# 팁 데이터 불러오기
```

```
tips = sns.load_dataset('tips')
```

```
# 전체 데이터에서 처음 5개의 row 데이터 표시 (내용 확인)
```

```
tips.head()
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4



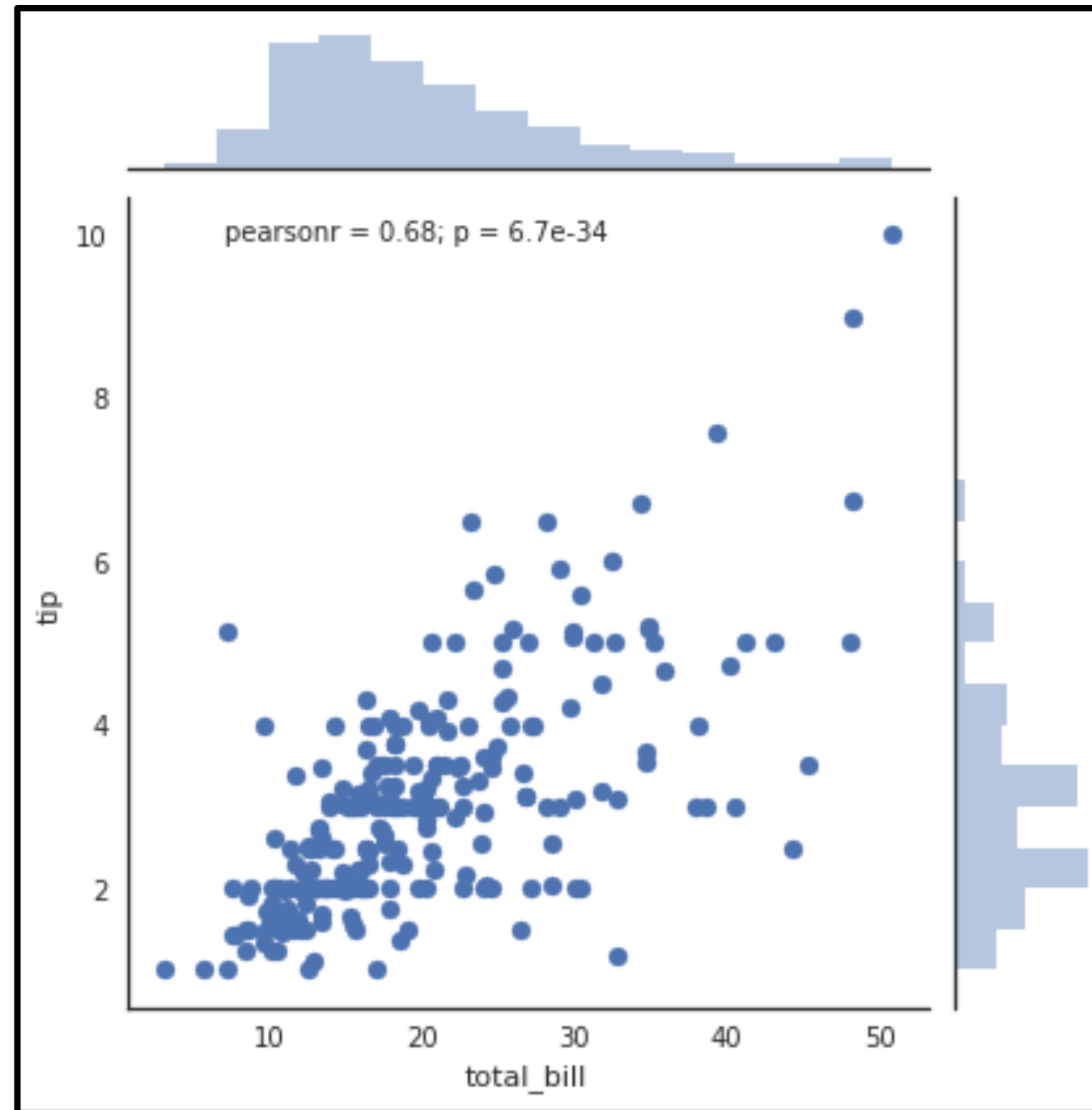
Seaborn 산점도와 히스토그램 그래프

```
sns.jointplot(x="total_bill", y="tip", data=tips)
plt.show()
```

- `jointplot(x, y, data, kind)`
 - 산점도와 히스토그램을 함께 출력
 - `x, y` = 그래프에 표시될 데이터의 x축, y축
 - `data` = 사용할 데이터프레임 지정
 - `kind` = 그래프의 종류 지정 (default=scatter)



Seaborn 산점도와 히스토그램 그래프



- kind를 지정하지 않고 기본값으로 출력한 결과
- 산점도와 히스토그램을 확인할 수 있음



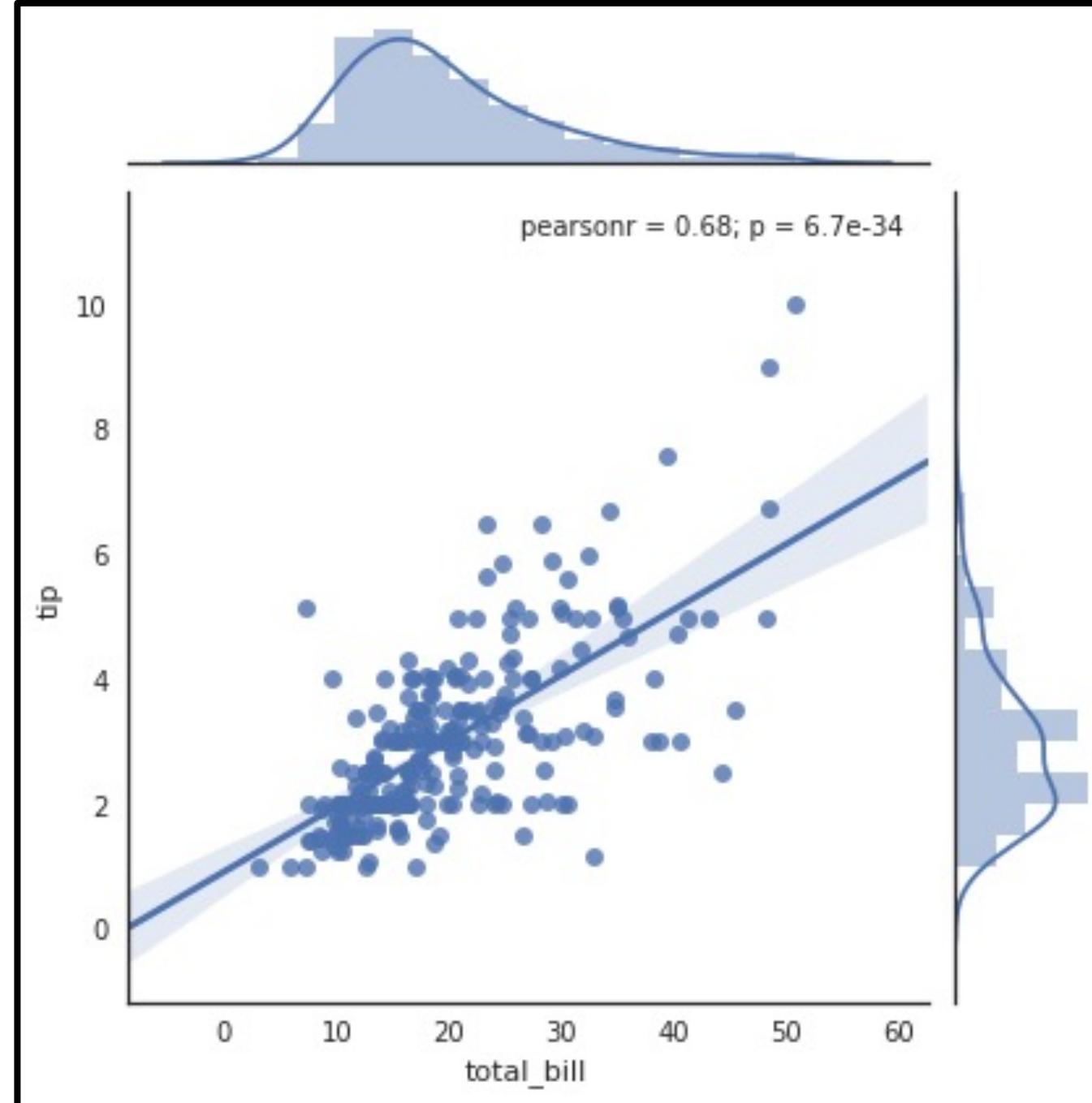
Seaborn 산점도와 히스토그램 그래프

```
# reg (regression + scatter) : 근사선  
sns.jointplot("total_bill", "tip", data=tips, kind="reg")  
plt.show()
```

- kind
 - 'scatter' – 산점도 (default)
 - 'reg' – 근사선 추가
 - 'resid' – 잔차플롯 추가
 - 'kde' – 커널 밀도 추정



Seaborn 산점도와 히스토그램 그래프



- kind에 reg를 지정한 출력 결과
- 근사선이 추가된 산점도와 히스토그램을 확인할 수 있음